

Explaining Machine Learning Models with Interactive Natural Language Conversations Using TalkToModel

Dylan Slack
UC Irvine
dslack@uci.edu

Satyapriya Krishna
Harvard University
skrishna@g.harvard.edu

Himabindu Lakkaraju*
Harvard University
hlakkaraju@hbs.edu

Sameer Singh*
UC Irvine / AI2
sameer@uci.edu

Abstract

Practitioners increasingly use machine learning (ML) models, yet they have become more complex and harder to understand. To address this issue, researchers have proposed techniques to explain model predictions. However, practitioners struggle to use explainability methods because they do not know which to choose and how to interpret the results. We address these challenges by introducing **TalkToModel: an interactive dialogue system that enables users to explain ML models through natural language conversations**. TalkToModel comprises **three** components: **1) an adaptive dialogue engine that interprets natural language and generates meaningful responses, 2) an execution component, which constructs the explanations used in the conversation, 3) a conversational interface**. In real-world evaluations, 73% of healthcare workers agreed they would use TalkToModel over existing systems for understanding a disease prediction model, and 85% of ML professionals agreed TalkToModel was easier to use, demonstrating that TalkToModel is highly effective for model explainability.

1 Introduction

Due to their strong performance, machine learning (ML) models increasingly make consequential decisions in several critical domains, such as healthcare, finance, and law. However, state-of-the-art ML models, such as deep neural networks, have become more complex and hard to understand. This dynamic poses challenges in real-world applications for model stakeholders who need to understand why models make predictions and whether to trust them. Consequently, practitioners have often turned to *inherently interpretable* machine learning models for these applications including decision lists and sets [33, 8, 75, 84] and generalized additive models [41, 5, 14, 13, 85], which people can more easily understand. Nevertheless, black-box models are often more flexible and accurate, motivating the development of *post-hoc* explanations that explain the predictions of trained ML models. These techniques either fit faithful models in the local region around a prediction or inspect internal model details, like gradients, to explain predictions [56, 66, 61, 67, 58, 29, 65, 68].

Yet, **recent work suggests practitioners often have difficulty using explainability techniques [35, 31, 79]**. These challenges are **due to difficulty figuring out which explanations to implement, how to interpret their results, and answering followup questions beyond the initial explanation**. In the past,

*Equal Contribution

researchers have proposed several point-and-click dashboard techniques to help overcome these issues, such as the Language Interpretability Tool (LiT) [73], which is designed to understand natural language processing (NLP) models and the "What-If" Tool [2]—a tool aimed at performing counterfactual analyses for models. However, these methods still require a high level of expertise because users must know which explanations to run and lack the flexibility to support arbitrary follow up questions users might have. Overall, being able to understand ML models through simple and intuitive interactions is a key bottleneck in adoption across many applications.

Natural language dialogues are a promising solution for supporting broad and accessible interactions with ML models due to their ease of use, capacity, and support for continuous discussion. However, designing a dialogue system that enables a satisfying model understanding experience introduces several challenges. First, the system must handle many conversation topics about the model and data while facilitating natural conversation flow [77]. For instance, these topics may include explainability questions like the most important features for predictions and general questions such as data statistics or model errors. Further, the system must work for a variety of model classes and data, and it should understand language usage across different settings [12]. For example, participants will use different terminology in conversations about loan prediction compared to disease diagnosis. Last, the dialogue system should generate accurate responses that address the users’ core questions [51, 86]. In the literature, researchers have suggested some prototype designs for generating explanations using natural language. However, these initial designs address specific explanations and model classes, limiting their applicability in general conversational explainability settings [69, 20].

In this work, we address these challenges by introducing TalkToModel: a system that enables open-ended natural language dialogues for understanding ML models for any tabular dataset and classifier (an overview of TalkToModel is provided in Figure 1). Users can have discussions with TalkToModel about why predictions occur, how the predictions would change if the data changes, and how to flip predictions, among many other conversation topics (an example conversation is provided in Table 1). Further, they can perform these analyses on any group in the data, such as a single instance or a specific group of instances. For example, on a disease prediction task, users can ask “how important is BMI for the predictions?” or “so how would decreasing the glucose levels by ten change the likelihood of men older than twenty having the disease?” TalkToModel will respond by describing how, for instance, BMI is the most important feature for predictions, and decreasing glucose will decrease the chance by 20%. From there, users can engage further in the conversation by asking follow up questions like, “what if you instead increased glucose by ten for that group of men?” and TalkToModel use the context to accurately respond. Conversations with TalkToModel make model explainability straightforward because users can simply talk with the system in natural language about the model, and the system will generate useful responses.

To support such rich conversations with TalkToModel, we introduce techniques for both language understanding and model explainability. First, we propose a *dialogue engine* that parses user text inputs (referred to as *user utterances*) into an SQL-like programming language using a large language model (LLM). The LLM performs the parsing by treating the task of translating user utterances into the programming language as a seq2seq learning problem, where the user utterances are the source and parses in the programming language are the targets [71]. In addition, the TalkToModel language combines operations for explanations, ML error analyses, data manipulation, and descriptive text into a single language capable of representing a wide-variety of potential conversation topics most model explainability needs (an overview of the different operations is provided in Table 2). To support the system adapting to any dataset and model, we introduce lightweight adaption techniques to fine-tune LLMs to perform the parsing, enabling strong generalization to new settings. Second, we introduce an *execution engine* that runs the operations in each parse. To reduce the burden of users deciding which explanations to run, we introduce methods that automatically select explanations for the user. In

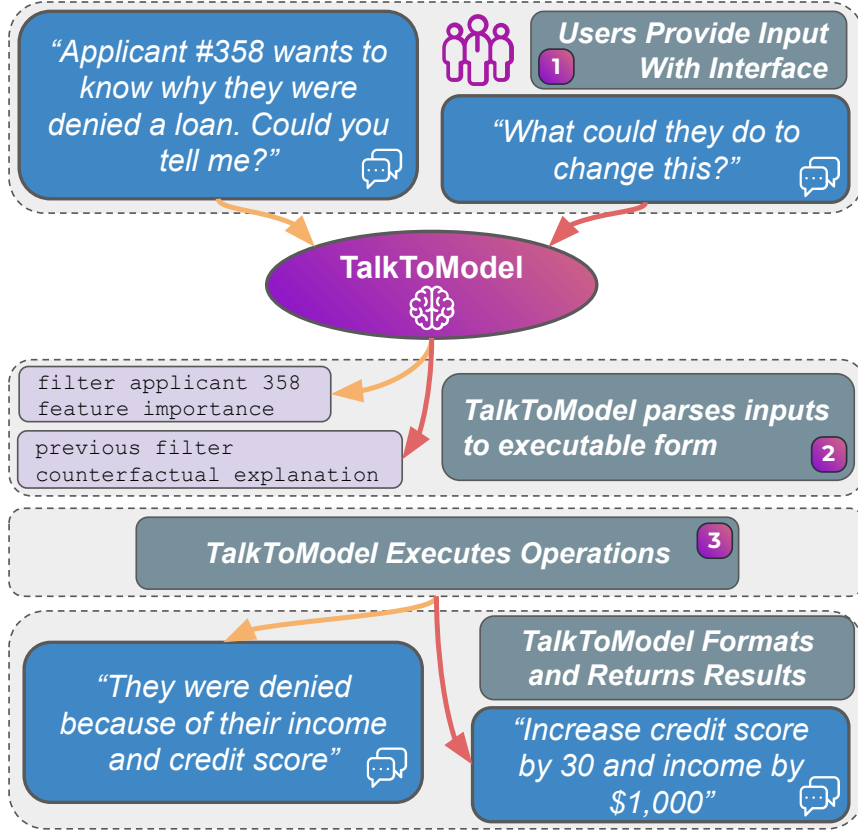


Figure 1: **Overview of TalkToModel:** Instead of writing code, users have conversations with TalkToModel as follows: (1) users supply natural language inputs. (2) the dialogue engine parses the input into an executable representation. (3) the execution engine runs the operations and the dialogue engine uses the results in its response.

particular, this engine runs many explanations, compares their fidelities, and selects the most accurate ones. Finally, we construct a *text interface* where users can engage in open-ended dialogues using the system, enabling anyone, including those with minimal technical skills, to understand ML models.

2 Results

In this section, we demonstrate TalkToModel accurately understands users in conversations by evaluating its language understanding capabilities on ground truth data. Next, we evaluate the effectiveness of TalkToModel for model understanding by performing a real-world human study on healthcare workers (e.g., doctors and nurses) and ML practitioners, where we benchmark TalkToModel against existing explainability systems. We find users both prefer and are more effective using TalkToModel than traditional point-and-click explainability systems, demonstrating its effectiveness for understanding ML models.

Language Understanding

Here, we quantitatively assess the language understanding capabilities of TalkToModel by creating gold parse datasets and evaluating the system’s accuracy on this data.

Table 1: A conversation about diabetes prediction, demonstrating the breadth of different conversation points the system can discuss.

User “utterances” & TalkToModel	response
“What are the predictions on patients older than 30? Could you also tell me what would happen to these predictions if we increased bmi by 10?”	On data with age greater than 30.0, the model predicts: ⇒ No Diabetes on 46.8% of data ⇒ Diabetes on 53.2% of data On data with age greater than 30.0 and bmi is increased by 10.0, the model predicts: ⇒ No Diabetes on 30.1% of data ⇒ Diabetes on 69.9% of data
“So how important is bmi for the model’s predictions on the over thirty group?”	For the model’s predictions on data with age greater than 30.0, the bmi feature is ranked on average 2.86 (95% CI [2.685, 3.035]). Here, rank 1 is the most important feature (out of 8 features). Overall, bmi is highly important for the model’s predictions.
“Ah, so which two features are mostly more important than bmi for these people?”	For the models predictions on data where age is greater than 30.0, the top 2 most important features are: 1. glucose 2. pedigree_function
“What types of patients is the model typically predicting incorrect here?”	For data with age greater than 30.0, the model typically predict predicts incorrect: ⇒ if (bmi > 26.95) and (glucose ≤ 125.0) and (pedigree_function then the model is incorrect on <i>62.3%</i> of <i>53</i> samples ⇒ (bmi > 26.95) and (glucose > 125.0) and (age > 57.5 ≤ 0.943) then the model is incorrect on <i>62.5%</i> of <i>8</i> samples ⇒ (bmi ≤ 26.95) and (glucose > 180.5) then the model is incorrect on <i>100.0%</i> of <i>2</i> samples. Want to take a closer look at these rules?

Gold Parse Collection We construct gold datasets (i.e., ground truth (utterance, parse) pairs) across multiple datasets to evaluate the language understanding performance of our models. To construct these gold datasets, we adopt an approach inspired by Yu et al. [83], which constructs a similar dataset for multitask semantic parsing.

Our gold dataset generation process is as follows. First, we write 50 (utterance, parse) pairs for the particular task (i.e., loan or diabetes prediction). These utterances range from simple “How likely are people in the data to have diabetes?” to complex “If these people were not unemployed, what’s the likelihood they are good credit risk? Why?” and conversational “What if they were twenty years older?”. We include each operation (Table 2) at least twice in the parses, to make sure there is good coverage. From there, we ask Mechanical Turk workers to rewrite the utterances while preserving their

Table 2: Overview of the *operations* supported by TalkToModel, which are incorporated into the conversation to generate responses.

	operation, arguments, and description
Data	<code>filter(dataset, feature, value, comparison)</code> : filters <code>dataset</code> by using value and comparison operator <code>change(dataset, feature, value, variation)</code> : Changes <code>dataset</code> by increasing, decreasing, or setting feature by <code>value</code> <code>show(list)</code> : Shows items in list in the conversation <code>statistic(dataset, metric, feature)</code> : Computes summary statistic for <code>feature</code> <code>count(list)</code> : Length of list <code>and(op1, op2)</code> : Logical “and” of two operations <code>or(op1, op2)</code> : Logical “or” of two operations
Explainability	<code>explain(dataset, method, class=predicted)</code> : Feature importances on <code>dataset</code> <code>cfe(dataset, number, class=opposite)</code> : Gets <code>number</code> counterfactual explanations <code>topk(dataset, k)</code> : Top <code>k</code> most important features <code>important(dataset, feature)</code> : Importance ranking of <code>feature</code> <code>interaction(dataset)</code> : Interaction effects between features <code>mistakes(dataset)</code> : Patterns in the model’s errors on <code>dataset</code>
ML	<code>predict(dataset)</code> : Model predictions on <code>dataset</code> <code>likelihood(dataset)</code> : Prediction probabilities on <code>dataset</code> <code>incorrect(dataset)</code> : Incorrect predictions <code>score(dataset, metric)</code> : Scores the model with <code>metric</code>
Conv.	<code>prev_filter(conversation)</code> : Gets last filters <code>prev_operation(conversation)</code> : Gets last non-filtering operations <code>followup(conversation)</code> : Respond to system followups
Description	<code>function()</code> : Overview of the system’s capabilities <code>data(dataset)</code> : Summary of dataset <code>model()</code> : Description of <code>model</code> <code>define(term)</code> : Defines <code>term</code>

semantic meaning to ensure that the ground truth parse for the revised utterance is the same but the phrasing differs. We ask workers to rewrite each pair 8 times for a total of 400 (utterance, parse) pairs per task. Next, we filter out low-quality mturk revisions. We ask the crowd sourced workers to rate the similarity between the original utterance and revised utterance on a scale of (1-4), where 4 indicates the utterances have the same meaning and 1 that they do not have the same meaning. We collect 5 ratings per revision and remove (utterance, parse) pairs that score below 3.0 on average. Finally, we perform an additional filtering step to ensure data quality by inspecting the remaining pairs ourselves and removing any bad revisions.

Since we want to evaluate TalkToModel’s capacity to generalize across different scenarios, we perform this data collection process across 3 different tasks: Pima Indian Diabetes Dataset [19], German Credit Dataset [19], and the COMPAS recidivism dataset [9]. After collecting revisions and ensuring quality, we are left with 200 pairs for German Credit, 190 for diabetes, and 146 for COMPAS.

Table 3: Exact Match Parsing Accuracy (%) for the 3 gold datasets, on the IID and Compositional splits, as well as Overall. The fine-tuned T5 models perform significantly better than few-shot GPT-J, and T5 Large performed the best. These results demonstrate that TalkToModel can understand user intentions with a high degree of accuracy using the T5 models.

	German			Compas			Diabetes		
	IID	Comp.	Overall	IID	Comp.	Overall	IID	Comp.	Overall
Nearest Neighbors	26.2	0.0	16.5	27.4	0.0	21.9	10.9	0.0	8.4
GPT-Neo 1.3B									
10-SHOT	41.3	4.1	27.5	35.9	0.0	28.8	40.1	7.0	32.6
20-SHOT	39.7	0.0	25.0	39.3	0.0	31.5	42.9	2.3	33.7
30-SHOT	42.9	0.0	27.0	39.3	0.0	31.5	41.5	4.7	33.2
GPT-Neo 2.7B									
5-SHOT	38.1	4.1	25.5	35.9	3.4	29.5	46.9	7.0	37.9
10-SHOT	38.1	6.8	26.5	40.2	3.4	32.9	40.8	9.3	33.7
20-SHOT	39.7	0.0	25.0	39.3	0.0	31.5	42.9	2.3	33.7
GPT-J 6B									
5-SHOT	51.6	14.9	38.0	51.3	6.9	42.5	55.8	7.0	44.7
10-SHOT	57.9	9.5	40.0	49.6	3.4	40.4	53.7	9.3	43.7
T5									
SMALL	61.1	32.4	50.5	71.8	10.3	59.6	77.6	30.2	66.8
BASE	68.3	48.6	61.0	65.0	10.3	54.1	84.4	34.9	73.2
LARGE	74.6	44.6	63.5	76.9	24.1	66.4	84.4	51.2	76.8

Models We compare two strategies for using pre-trained LLMs to parse user utterances into the grammar **1.)** few-shot GPT-J [76] and **2.)** finetuned T5 [53]. Both these models translate user utterances into the TalkToModel grammar in a seq2seq fashion. However, the GPT-J models are higher-capacity and more amenable to be trained by in-context learning. This procedure includes examples of the input and target from the training prepended to the test instance [10, 45, 80]. On the other hand, the T5 models require traditional finetuning on the input and target pairs. Consequently, the few-shot approach is quicker to set up because it does not require finetuning, making it easier for users to get started with the system. However, the finetuned T5 leads to improved performance and a better user experience overall while taking longer to set up. To train these models through finetuning or prompting, we generate synthetic (utterance, parse) pairs because it is impractical to assume that we can collect ground truth pairs for every new task we wish to use TalkToModel. We provide additional training details in the methods section.

We evaluate both fine-tuned T5 models and few-shot GPT-J models on the testing data. We additionally implement a naive nearest neighbors baseline, where we select the closest user utterance in the synthetic training set according to cosine distance of `all-mpnet-base-v2` sentence embeddings and return the corresponding parse [54]. For the GPT-J models, we compare N -Shot performance, where N is the number of (utterance, parse) pairs from the synthetically generated training sets included in the prompt, and sweep over a range of N for each model. For the larger models, we have to use relatively smaller N in order for inference to fit on a single 48GB GPU.

When parsing the utterances, one issue is that their generations are unconstrained and may generate parses outside the grammar, resulting in the system failing to run the parse and bad user experiences. To ensure the generations are grammatical, we constrain the decodings to be in the grammar [63]. This

technique, referred to as *guided decoding*, constrains the LLM generations to only allow those tokens that appear next in the grammar at any point during generation. Practically, we accomplish this by recompiling the grammar at inference time into an equivalent grammar consisting of the tokens in the LLM’s vocabulary. While decoding from the LLM, we fix the likelihood of ungrammatical tokens to 0 at every generation step. Thus, the LLM only generates grammatical parses.

Evaluating The Parsing Accuracy To evaluate performance on the datasets, we use the exact match parsing accuracy [72, 83, 28]. This metric is whether the parse exactly matches the gold parse in the dataset. In addition, we perform the evaluation on two splits of each gold parse dataset, in addition to the overall dataset. These splits are the IID and compositional splits. The IID split contains (utterance, parse) pairs where the parse’s **operations** and their structure (but not necessarily the arguments) are in the training data. The compositional split consists of the remaining parses that are not in the training data. Because LM’s struggle compositionally, this split is generally much harder for LM’s to parse [48, 82].

Accuracy We present the results in Table 3. The fine-tuned T5 performs better overall than the few shot GPT-J models. In particular, the T5 Large models perform strongly on both the IID and compositional data and can even parse complex compositional phrases. Notably, the T5 small model performs better than the GPT-J 6B model, which has two orders of magnitude more parameters. This dynamic is particularly true in the compositional splits in the data where the GPT-J few shot models never exceed 10% parsing accuracy. Overall, these results indicate TalkToModel can understand user utterances with a high degree of accuracy using our best performing T5 models. Further, we recommend using this model for the best results and use it for our remaining evaluation.

User Study: Utility of Explainability Dialogues

The results in the previous subsection show TalkToModel understands user intentions to a high degree of accuracy. In this subsection, we evaluate how well the end-to-end system helps users understand ML models compared to current explainability systems.

Study Overview We compare TalkToModel against *explainerdashboard*, one of the most popular open-source explainability dashboards [18]. This dashboard has similar functionality to TalkToModel, considering it provides an accessible way to compute explanations and perform model analyses. Thus, it is a reasonable baseline. Last, we perform this comparison using the Diabetes dataset, and a gradient boosted tree trained on the data [50]. To compare both systems in a controlled manner, we ask participants to answer general ML questions with TalkToModel and the dashboard. Each question is about basic explainability and model analysis, and participants answer using multiple choice, where one of the options is “Could not determine.” if they cannot figure out the answer (though it is straightforward to answer all the questions with both interfaces). For example, questions are about comparing feature importances “Is glucose more important than age for the model’s predictions for data point 49?” or model predictions “How many people are predicted not to have diabetes but do not actually have it?” Participants answer 10 total questions. We divide the 10 questions into 2 blocks of 5 questions each. Both blocks have similar questions but different values to control for memorization (exact questions given in Supplementary Information A). Participants use TalkToModel to answer one block of questions and the dashboard for the other block. In addition, we provide a tutorial on how to use both systems before showing users the questions for the system. Last, we randomize question, block, and interface order to control for biases due to showing interfaces or questions first.

Table 4: User study results: % of respondents that agree ($>$ Neutral Likert score) TalkToModel is better than the dashboard in the 4 comparison questions. A significant portion of respondents agreed TalkToModel is better than the dashboard in all the categories except Grad. students and “Likeliness To Use”. Still, a majority agreed TalkToModel was superior in this case.

Comparison	% Agree TalkToModel Better	
	Health Care Workers	ML Grad. Students
Easiness	82.2	84.6
Confidence	77.7	69.2
Speed	84.4	84.6
Likeliness To Use	73.3	53.8

Metrics Following previous work on evaluating human and ML coordination and trust, we assess several metrics to evaluate user experiences [17, 21, 24]. We evaluate the following statements along 1-7 Likert scale at the end of the survey:

- **Easiness:** *I found the conversational interface easier to use than the dashboard interface*
- **Confidence:** *I was more confident in my answers using the conversational interface than the dashboard interface*
- **Speed:** *I felt that I was able to more rapidly arrive at an answer using the conversational interface than the dashboard interface*
- **Likeliness To Use:** *Based on my experience so far with both interfaces, I would be more likely to use the conversational interface than the dashboard interface in the future*

To control for bias associated with the ordering of the terms conversational interface and dashboard interface, we randomized their ordering. We also measure accuracy and time-taken to answer each question. Last, we asked to participants to write a short description comparing their experience with both interfaces to capture participants qualitative feedback about both systems.

Recruitment Since TalkToModel provides an accessible way to understand ML models, we expect it to be useful for subject matter experts with a variety of experience in ML, including users without any ML experience. As such, we recruited 45 English speaking healthcare workers to take the survey using the Prolific service [49] with minimal or no ML expertise. This group comprises a range of healthcare workers, including doctors, pharmacists, dentists, psychiatrists, healthcare project managers, and medical scribes. The vast majority of this group (43) stated they had either no experience with ML or had heard about it from reading articles online, while two members indicated they had equivalent to an undergraduate course in ML. As another point of comparison, we recruited ML professionals with relatively higher ML expertise from ML Slack channels and email lists. We received 13 potential participants, all of which had graduate course level ML experience or higher, and included all of them in the study. We received IRB approval for this study from our institution’s IRB approval process and informed consent from participants.

Metric Results A significant majority of health care workers agreed they preferred TalkToModel in all the categories we evaluated (Table 4). The same is true for the ML professionals, save for whether they were more likely to use TalkToModel in the future, where 53.8% of participants agreed they would instead use TalkToModel in the future. In addition, participants subjective notions around how

Table 5: User study results: Completion rate and accuracy across interfaces and participant groups. We compute the completion rate as the questions users provided and answer for and did not mark “could not determine.” We measure accuracy on completed questions. Participants answered questions at a higher rate more accurately using TalkToModel than the dashboard.

	% Questions Completed		% Accuracy On Completed Questions	
	Dash.	TalkToModel	Dash.	TalkToModel
Health Care Workers	74.7	86.2	66.1	91.8
ML Grad. Students	73.8	93.9	62.5	100.0

quickly they could use TalkToModel aligned with their actual speed of use, and both groups arrived at answers using TalkToModel significantly quicker than the dashboard. The median question answer time (measured at the total time taken from seeing the question to submitting the answer) using TalkToModel was 76.3 seconds, while it was 158.8 seconds using the dashboard.

Participants were also much more accurate and completed questions at a higher rate (i.e., they did not mark “could not determine”) using TalkToModel (Table 5). While both health care workers and ML practitioners clicked could not determine for a quarter of the questions using the dashboard, this was true for 13.8% of health care workers and 6.1% of ML professionals using TalkToModel, demonstrating the usefulness of the conversational interface. On completed questions, both groups were much more accurate using TalkToModel than the dashboard. Most surprisingly, though ML professionals agreed they preferred TalkToModel only about half the time, they answered all the questions correctly using it, while they only answered 62.5% of questions correctly with the dashboard. Finally, we observed TalkToModel’s conversational capabilities were highly effective. There were only 6 utterances out of over 1,000 total utterances the conversational aspect of the system failed to resolve. These failure cases generally involved certain discourse aspects like asking for additional elaboration (“more description”).

The largest source of errors for participants using the explainability dashboard were two questions concerning the top most important features for individual predictions. The errors for these questions account for 47.4% of health care workers and 44.4% of ML professionals’ total mistakes. Answering these questions with the dashboard requires users to perform multiple steps, including choosing the feature importance tab in the dashboard, selecting local explanations for the correct instance, and ranking the features according to their importance. On the other hand, the streamlined text interface of TalkToModel made it much simpler to solve these questions resulting fewer errors.

Qualitative Results For the qualitative user feedback, we provide representative quotes from similar themes in the responses. Users expressed that they could more rapidly and easily arrive at results, which could be helpful for their professions,

I prefer the conversational interface because it helps arrive at the answer very quickly. This is very useful especially in the hospital setting where you have hundreds of patients getting check ups and screenings for diabetes because it is efficient and you can work with medical students on using the system to help patient outcomes.—P39 medical worker at a tertiary hospital.

Participants also commented on the user friendliness of TalkToModel and its strong conversational capabilities, stating, “the conversational [interface] was straight to the point, way easier to use”—P35 Nurse, and that “the conversational interface is hands-down much easier to use... it feels like one

is talking to a human.”—P45 ML Professional. We did not find any negative feedback surrounding the conversational capabilities of the system. Users also commented on how easy it was to access information compared to the dashboard,

With the conversational interface you can ask whatever you want to know and with the dashboard you need to specifically search information that you don’t actually know where it is.—P31 Physical Therapist.

All in all, users expressed strong positive sentiment about TalkToModel due to the quality of conversations, presentation of information, accessibility, and speed of use.

Several ML professionals brought up points that could serve as future research directions. Notably, participants stated they would rather look at the data themselves rather than rely on an interface that rapidly provides an answer,

I would almost always rather look at the data myself and come to a conclusion than getting an answer within seconds.—P11 ML Professional.

In the future, it would be worthwhile including visualizations of raw data and analyses performed by the system to increase trust with expert users, such as ML professionals, who may be skeptical of the high-level answers provided by the system currently.

3 Discussion

With ML models increasingly becoming more complex, there is need to develop techniques to explain model predictions to stakeholders. Nevertheless, it is often the case that practitioners struggle to use explanations and often have many follow up questions they wish to answer. In this work, we show TalkToModel makes explainable AI accessible to users that come from a range of backgrounds by using natural language conversations. Our experimental findings demonstrate TalkToModel both comprehends users to a high-degree of accuracy and can help users understand the predictions of ML models much better than existing systems. In particular, we showed TalkToModel is a highly effective way for domain experts such as healthcare workers to understand ML models, like those applied to disease diagnosis. Last, we designed TalkToModel to be highly extensible and release the code, data, and a demo for the system at <https://github.com/dylan-slack/TalkToModel>, making it straightforward for explainability users and researchers to build on the system.

In the future, it will be helpful to investigate applications of TalkToModel *in-the-wild*, such as in doctors’ offices, laboratories, or professional settings, where model stakeholders use the system to understand their models. In addition, it will also be helpful to explore how to use language models to generate conversation responses grounded in the results of the operations. Finally, it will also be helpful to evaluate how best to integrate TalkToModel in existing scientific and professional work streams to promote its impact and usefulness.

4 Methods

In this section, we describe the components of TalkToModel. First, we introduce the dialogue engine and discuss how it understands user inputs, maps them to operations, and generates text responses based on the results of running the operations. Second, we describe the execution engine, which runs the operations. Finally, we provide an overview of the interface and the extensibility of TalkToModel.

4.1 Text Understanding

To understand the intent behind user utterances, the system learns to translate or *parse* them into logical forms. These parses represent the intentions behind user utterances in a highly-expressive and structured programming language TalkToModel executes.

Compared to dialogue systems that execute specific tasks by modifying representations of the internal state of the conversation [15, 37], our parsing-based approach allows for more flexibility in the conversations, supporting open-ended discovery, which is critical for model understanding. Also, this strategy produces a structured representation of user utterances instead of open-ended systems that generate unstructured free text [60]. Having this structured representation of user inputs is key for our setting where we need to execute specific operations depending on the user’s input, which would not be straightforward with unstructured text.

TalkToModel performs the following steps to accomplish this: **1)** the system constructs a grammar for the user-provided dataset and model, which defines the set of acceptable parses, **2)** TalkToModel generates (utterance, parse) pairs for the dataset and model, **3)** the system finetunes a large language model (LLM) to translate user utterances into parses, and **4)** the system responds conversationally to users by composing the results of the executed parse into a response that provides context for the results and opportunities to follow up.

Grammar To represent the intentions behind the user utterances in a structured form, TalkToModel relies on a grammar, defining a domain specific language for model understanding. While the user utterances themselves will be highly diverse, the grammar creates a way to express user utterances in a structured yet highly expressive fashion that the system can reliably execute. Compared with approaches that treat determining user intentions in conversations as a classification problem [39, 11], using a grammar enables the system to express compositions of operations and arguments that take on many different values, such as real numbers, that would otherwise be combinatorially impossible in a prediction setting. Instead, TalkToModel translates user utterances into this grammar in a seq2seq fashion, overcoming these challenges [71]. This grammar consists of production rules that include the operations the system can run (an overview is provided in Table 2), the acceptable arguments for each operation, and the relations between operations. One complication is that user-provided datasets have different feature names and values, making it hard to define one shared grammar between datasets. Instead, we update the grammar based on the feature names and values in a new dataset. For instance, if a dataset only contained the feature names `age` and `income`, these two names would be the only acceptable values for the `feature` argument in the grammar.

To ensure our grammar provides sufficient coverage for `XAI` questions, we vary our grammar supports the questions from the *XAI question bank*. This question bank was introduced by Liao et al. [38] based on interviews with AI product designers and includes 31 core, prototypical questions XAI systems should answer, excluding socio-technical questions beyond the scope of TalkToModel (e.g., What are the results of other people using the [model]). The prototypical questions address topics such as the input/data to the model (“What is the distribution of a given feature?”), model output (“What kind of output does the system give?”), model performance (“How accurate are the predictions?”), global model behavior (“What is the systems overall logic?”), why/why not the system makes individual predictions (“Why is this instance given this prediction?”), and what-if or counterfactual questions (“What would the system predict if this instance changes to...?”). To evaluate how well TalkToModel covers these questions, we review each question and evaluate whether our grammar can parse it. Overall, we find our grammar supports 30/31 of the prototypical questions. We provide a table of each question and corresponding parse in Supplementary Table 6 and Supplementary Table 7. Overall, the grammar covers the vast majority of XAI related questions, and therefore, has good coverage of XAI topics.

Supporting Context In Dialogues User conversations with TalkToModel naturally include complex conversational phenomena such as anaphora and ellipsis [26, 74, 27]. Meaning, conversations refer back to events earlier in the conversation (“what do you predict for *them*?”) or omit information that must be inferred from conversation (“Now show me for people predicted incorrectly.”). However, current language models only parse a single input, making it hard to apply them in settings where the context is important. To support context in the dialogues, TalkToModel introduces on a set of operations in the grammar that determine the context for user utterances. In contrast with approaches that maintain the conversation state using neural representations [15, 23], grammar operations allow for much more trustworthy and dependable behavior while still fostering rich interactions, which is critical for high-stakes settings, and similar mechanisms for incorporating grammar predicates across turns have been shown to achieve strong results [27]. In particular, we leverage two operations: **previous filter** and **previous operation**, which look back in the conversation to find the last filter and last operation, respectively. These operations also act recursively. Therefore, if the last filter is a **previous filter** operation, TalkToModel will recursively call **previous filter** to resolve the entire stack of filters. As a result, TalkToModel is capable of addressing instances of anaphora and ellipsis by using these operations to resolve the entity via co-reference or infer it from the previous conversation history. This dynamic enables users to have complex and natural conversations with TalkToModel.

Parsing Dataset Generation To parse user utterances into the grammar, we finetune an LLM to translate utterances into the grammar in a seq2seq fashion. We use LLMs because these models have been trained on large amounts of text data and are solid priors for language understanding tasks. Thus, they can better understand diverse user inputs than training from scratch, improving the user experience. Further, we automate the finetuning of an LLM to parse user utterances into the grammar by generating a training dataset of (utterance, parse) pairs. Compared to dataset generation methods that use human annotators to generate and label datasets for training conversation models [22, 59], this approach is much less costly and time consuming, while still being highly effective, and supports users getting conversations running very quickly. This strategy consists of writing an initial set of user utterances and parses, where parts of the utterances and parses are *wildcard* terms. TalkToModel enumerates the wildcards with aspects of a user-provided dataset, such as the feature names, to generate a training dataset. Depending on the user-provided dataset schema, TalkToModel typically generates anywhere from 20,000-40,000 pairs. Last, we have already written the initial set of utterances and parses, so users only need to provide their dataset to setup a conversation.

Semantic Parsing Here, we provide additional details about the semantic parsing approach for translating user utterances into the grammar. The two strategies for parsing user utterances using pre-trained LLMs that we considered were **1.)** few-shot GPT-J [76] and **2.)** finetuned T5 [53]. With respect to the few-shot models, because the LLM’s context window only accepts a fixed number of inputs, we introduce a technique to select the set of most relevant prompts for the user utterance. In particular, we embed all the utterances and identify the closest utterances to the user utterance according to the cosine distance of these embeddings. To ensure a diverse set of prompts, we only select one prompt per template. We prompt the LLM using these (utterance, parse) pairs, ordering the closest pairs immediately before the user utterance because LLMs exhibit recency biases [87]. Using this strategy, we experiment with the number of prompts included in the LLM’s context window. In practice, we use the **all-mpnet-base-v2** sentence transformer model to perform the embeddings [54], and we consider the GPT-J 6B, GPT-Neo 2.7B, and GPT-Neo 1.3B models in our experiments.

We also fine-tune pre-trained T5 models in a seq2seq fashion on our datasets. To perform fine-tuning, we split the dataset using a 90/10% train/validation split and train for 20 epochs to maximize the next token likelihood with a batch size of 32. We select the model with the lowest validation loss

at the end of each epoch. We fine-tune with a learning rate of $1e-4$ and the AdamW optimizer [40]. Last, our experiments consider the T5 Small, Base, and Large variants.

Generating Responses After TalkToModel executes a parse, it composes the results of the operations into a natural language response it returns to the user. TalkToModel generates these responses by filling in templates associated with each operation based on the results. The responses also include sufficient context to understand the results and opportunities for following up (examples in Table 1). Further, because the system runs multiple operations in one execution, TalkToModel joins response templates, ensuring semantic coherence, into a final response and shows it to the user. Compared to approaches that generate responses using neural methods [62], this approach ensures the responses are trustworthy and do not contain useless information hallucinated by the system, which would be a very poor user experience for the high-stakes applications we consider. Further, because TalkToModel supports a wide variety of different operations, this approach ensures sufficient diversity in responses, so they are not repetitive.

4.2 Executing Parses

In this subsection, we provide an overview of the execution engine, which runs the operations necessary to respond to user utterances in the conversation. Further, this component automatically selects the most faithful explanations for the user, helping ensure explanation accuracy.

Feature Importance Explanations At its core, TalkToModel explains why the model makes predictions to users with feature importance explanations. Feature importance explanations $\Phi(\mathbf{x}, f) \rightarrow \phi$ accept a data point $\mathbf{x} \in \mathbb{R}^d$ with d features and model as input $f(\mathbf{x}) \rightarrow \mathbf{y}$, where $\mathbf{y} \in [0, 1]$ is the probability for a particular class, and generates a feature attribution vector $\phi \in \mathbb{R}^d$, where greater magnitudes correspond to higher importance features [66, 68, 56, 81, 16, 6].

We implement the feature importance explanations using *post-hoc* feature importance explanations. Post-hoc feature importance explanations do not rely on internal details of the model f (e.g., internal weights or gradients) and only on the input data $\mathbf{x}$ and predictions $\mathbf{y}$ to compute explanations, so users are not limited to only certain types of models [57, 43, 34, 52, 36]. Note that our system can easily be extended to other explanations that rely on internal model details, if required [61, 47, 5, 70].

Explanation Selection While there exists several post hoc explanation methods, each one adopts a different definition of what constitutes an explanation [32]. For example, while LIME, SHAP, and Integrated Gradients all output feature attributions, LIME returns coefficients of local linear models, SHAP computes Shapley values, and Integrated Gradients leverages model gradients. Consequently, we automatically select the *most faithful* explanation for users, unless a user specifically requests a certain technique. Following prior works, we compute faithfulness by perturbing the most important features and evaluating how much the prediction changes [44]. Intuitively, if the feature importance ϕ correctly captures the feature importance ranking, perturbing more important features should lead to greater effects.

While previous works [43, 30], compute the faithfulness over many different thresholds, making comparisons harder, or require retraining entirely from scratch, we introduce a single metric that captures the prediction sensitivity to perturbing certain features called the *fudge score*. This metric is the mean absolute difference between the model’s prediction on the original input and a fudged version

on $\mathbf{m} \in \{0, 1\}^d$ features,

$$\text{Fudge}(f, \mathbf{x}, \mathbf{m}) = \frac{1}{N} \sum_{n=1}^N |f(\mathbf{x}) - f(\mathbf{x} + \epsilon_n \odot \mathbf{m})| \quad (1)$$

where $\odot$ is the tensor product and $\epsilon \sim \mathcal{N}(0, I\sigma)$ is $N \times d$ dimensional Gaussian noise. To evaluate faithfulness for a particular explanation method, we compute area under the fudge score curve on the top-k most important features, thereby summarizing the results into a single metric,

$$\mathbb{1}(k, \phi) = \begin{cases} 1 & \text{if } \phi_i \in \arg \max_{\phi' \subset \{1..d\}, |\phi'|=k} \sum_{i \in \phi'} |\phi_i| \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

$$\text{Faith}(\phi, f, \mathbf{x}, K) = \sum_{k=1}^K \text{Fudge}(f, \mathbf{x}, \mathbb{1}(k, \phi)) \quad (3)$$

where $\mathbb{1}(k, \phi)$ is the indicator function for the top-k most important features. Intuitively, if a set of feature importances ϕ correctly identifies the most important features, perturbing them will have greater effects on the model’s predictions, resulting in higher faithfulness scores. We compute faithfulness for multiple different explanations and select the highest. In practice, we consider LIME [57] with the following kernel widths [0.25, 0.50, 0.75, 1.0] and KernelSHAP [42]. We leave all settings to default besides the kernel widths for LIME. In practice, we set $\sigma = 0.05$ to ensure perturbations happen in the local region around the prediction, K to $\text{floor}(\frac{d}{2})$, and $N = 10,000$ to sample sufficiently. One complication arises for categorical features, where we cannot apply Gaussian perturbations. For these features, we randomly sample these features from a value in the dataset column 30% of the time to guarantee the feature remains categorical under perturbation. Last, if multiple explanations return similar fidelities, we use the explanation stability metric proposed by Alvarez-Melis and Jaakkola [7] to break ties, because it is much more desirable for the explanation to robust to perturbations [66, 3]. In order to use the *stability* metric proposed by Alvarez-Melis and Jaakkola [7] to break ties if the explanations fidelities are quite close (less than $\delta = 0.01$), we compute the jaccard similarity between feature rankings instead of the $l2$ norm as is used in their work. The reason is that the norm might not be comparable between explanation types, because they have different ranges, while the jaccard similarity should not be affected. Further, we compute the area under the top k curve using the jaccard similarity stability metric, as in Equation 3, to make this measure more robust.

Additional Explanation Types Since users will have explainability questions that cannot be answered solely with feature importance explanations, we include additional explanations to support a wider array of conversation topics. In particular, we support counterfactual explanations and feature interaction effects. These methods enable conversations about *how* to get different outcomes and if features *interact* with each other during predictions, supporting a broad set of user queries. We implement counterfactual explanations using DiCE, which generates a diverse set of counterfactuals [46]. Having access to many plausible counterfactuals is desirable because it enables users to see a breadth of different, potentially useful, options. Also, we implement feature interaction effects using the partial dependence based approach from Greenwell et al. [25] because it is effective and quick to compute.

Exploring Data and Predictions Because the process of understanding models often requires users to inspect the model’s predictions, errors, and the data itself, TalkToModel supports a wide variety of data and model exploration tools. For example, TalkToModel provides options for filtering data and performing what-if analyses, supporting user queries that concern subsets of data or what would happen if data points change. Users can also inspect model errors, predictions, prediction probabilities, compute summary statistics, and evaluation metrics for individuals and groups of instances.

TalkToModel additionally supports summarizing common patterns in mistakes on groups of instances by training a shallow decision tree on the model errors in the group. Also, TalkToModel enables descriptive operations, which explain how the system works, summarize the dataset, and define terms to help users understand how to approach the conversation. Overall, TalkToModel supports a rich set of conversation topics in addition to explanations, making the system a complete solution for the model understanding requirements of end users.

Extensibility

While we implement TalkToModel with several different choices for operations such as feature importance explanations and counterfactual explanations, TalkToModel is highly modular and system designers can easily incorporate new operations or change existing ones by modifying the grammar to best support their user populations. This design makes TalkToModel straightforward to extend to new settings, where different operations may be desired.

Data Availability

The German, Compas, and Diabetes datasets and models can be found at <https://github.com/dylan-slack/TalkToModel/tree/main/data>. The finetuned language models used for TalkToModel for each of these datasets can be found at <https://huggingface.co/dslack/all-finetuned-ttm-models>. The mturk generated dataset used to assess parsing accuracy and the accuracy results can be found at https://github.com/dylan-slack/TalkToModel/tree/main/experiments/parsing_accuracy. The user study response data is provided at <https://github.com/dylan-slack/TalkToModel/blob/main/data/ttm-user-study-responses.csv>.

Code Availability

We release an open source implementation of TalkToModel at <https://github.com/dylan-slack/TalkToModel>. The DOI is <https://doi.org/10.5281/zenodo.7502206>. Beyond the methods described so far, this release includes visualizations for conversations, interactive tooling to help users construct questions, saving results and conversation environments so they can be shared, abstractions for creating new operations and synthetic datasets, routines to adapt TalkToModel to new models and datasets automatically, and runtime optimizations (generating responses typically takes < 2 seconds).

References

- [1] Free google slides and powerpoint templates to boost your presentations. URL <https://slidesgo.com/>.
- [2] What-if tool. <https://pair-code.github.io/what-if-tool/>.
- [3] C. Agarwal, N. Johnson, S. K. Martin Pawelczyk, E. Saxena, M. Zitnik, and H. Lakkaraju. Rethinking stability for attribution-based explanations. *arXiv*, 2022.
- [4] C. Agarwal, S. Krishna, E. Saxena, M. Pawelczyk, N. Johnson, I. Puri, M. Zitnik, and H. Lakkaraju. OpenXAI: Towards a transparent evaluation of model explanations. In *Thirty-sixth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2022. URL <https://openreview.net/forum?id=MU2495w47rz>.

- [5] R. Agarwal, L. Melnick, N. Frosst, X. Zhang, B. Lengerich, R. Caruana, and G. E. Hinton. Neural additive models: Interpretable machine learning with neural nets. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 4699–4711. Curran Associates, Inc., 2021. URL <https://proceedings.neurips.cc/paper/2021/file/251bd0442dfcc53b5a761e050f8022b8-Paper.pdf>.
- [6] S. Agarwal, S. Jabbari, C. Agarwal, S. Upadhyay, S. Wu, and H. Lakkaraju. Towards the unification and robustness of perturbation and gradient based explanations. In M. Meila and T. Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 110–119. PMLR, 18–24 Jul 2021. URL <https://proceedings.mlr.press/v139/agarwal21c.html>.
- [7] D. Alvarez-Melis and T. S. Jaakkola. On the robustness of interpretability methods. *ICML Workshop on Human Interpretability in Machine Learning*, 2018.
- [8] E. Angelino, N. Larus-Stone, D. Alabi, M. Seltzer, and C. Rudin. Learning certifiably optimal rule lists for categorical data. *arXiv preprint arXiv:1704.01701*, 2017.
- [9] J. Angwin, J. Larson, S. Mattu, and L. Kirchner. Machine bias. In *ProPublica*, 2016.
- [10] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020. URL <https://proceedings.neurips.cc/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf>.
- [11] W. Cai and L. Chen. Predicting user intents and satisfaction with dialogue-based conversational recommendations. In *Proceedings of the 28th ACM Conference on User Modeling, Adaptation and Personalization*, UMAP ’20, page 33–42, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450368612. doi: 10.1145/3340631.3394856. URL <https://doi.org/10.1145/3340631.3394856>.
- [12] G. Carenini, V. O. Mittal, and J. D. Moore. Generating patient-specific interactive natural language explanations. *Proc Annu Symp Comput Appl Med Care*, pages 5–9, 1994.
- [13] C.-H. Chang, S. Tan, B. Lengerich, A. Goldenberg, and R. Caruana. How interpretable and trustworthy are gam’s? In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, pages 95–105, 2021.
- [14] C.-H. Chang, R. Caruana, and A. Goldenberg. Node-gam: Neural generalized additive model for interpretable deep learning. In *International Conference on Learning Representations*, 2022.
- [15] H. Chen, X. Liu, D. Yin, and J. Tang. A survey on dialogue systems: Recent advances and new frontiers. *SIGKDD Explor. Newsl.*, 19(2):25–35, nov 2017. ISSN 1931-0145. doi: 10.1145/3166054.3166058. URL <https://doi.org/10.1145/3166054.3166058>.
- [16] J. Chen, L. Song, M. J. Wainwright, and M. I. Jordan. L-shapley and c-shapley: Efficient model interpretation for structured data. In *International Conference on Learning Representations*, 2019.

- [17] Q. Chen, T. Schnabel, B. Nushi, and S. Amershi. Hint: Integration testing for ai-based features with humans in the loop. In *International Conference on Intelligent User Interfaces*. ACM, March 2022. URL <https://www.microsoft.com/en-us/research/publication/hint-integration-testing-for-ai-based-features-with-humans-in-the-loop/>.
- [18] O. Dijk, oegesam, R. Bell, Lily, Simon-Free, B. Serna, rajgupt, yanhong-zhao ef, A. Gädke, Hugo, and T. Okumus. oegedijk/explainerdashboard: v0.3.8.2: reverses set_shap_values bug introduced in 0.3.8.1. Apr. 2022. doi: 10.5281/zenodo.6408776. URL <https://doi.org/10.5281/zenodo.6408776>.
- [19] D. Dua and C. Graff. UCI machine learning repository, 2017. URL <http://archive.ics.uci.edu/ml>.
- [20] N. Feldhus, A. M. Ravichandran, and S. Möller. Mediators: Conversational agents explaining nlp model behavior. *arXiv preprint arXiv:2206.06029*, 2022.
- [21] M. Freed, J. Carbonell, G. Gordon, J. Hayes, B. A. Myers, D. Siewiorek, S. Smith, A. Steinfeld, and A. Tomasic. Radar: A personal assistant that learns to reduce email overload. In *Proceedings of 23rd National Conference on Artificial Intelligence*. AAAI, 2008.
- [22] J. Gao, M. Galley, and L. Li. Neural approaches to conversational AI. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics: Tutorial Abstracts*, pages 2–7, Melbourne, Australia, July 2018. Association for Computational Linguistics. doi: 10.18653/v1/P18-5002. URL <https://aclanthology.org/P18-5002>.
- [23] S. Gao, A. Sethi, S. Agarwal, T. Chung, and D. Hakkani-Tur. Dialog state tracking: A neural reading comprehension approach. In *Proceedings of the 20th Annual SIGdial Meeting on Discourse and Dialogue*, pages 264–273, Stockholm, Sweden, Sept. 2019. Association for Computational Linguistics. doi: 10.18653/v1/W19-5932. URL <https://aclanthology.org/W19-5932>.
- [24] A. Glass, D. L. McGuinness, and M. Wolverton. Toward establishing trust in adaptive agents. In *Proceedings of the 13th International Conference on Intelligent User Interfaces*, IUI ’08, page 227–236, New York, NY, USA, 2008. Association for Computing Machinery. ISBN 9781595939876. doi: 10.1145/1378773.1378804. URL <https://doi.org/10.1145/1378773.1378804>.
- [25] B. M. Greenwell, B. C. Boehmke, and A. J. McCarthy. A simple and effective model-based variable importance measure. *arXiv*, abs/1805.04755, 2018.
- [26] B. J. Grosz, A. K. Joshi, and S. Weinstein. Providing a unified account of definite noun phrases in discourse. In *21st Annual Meeting of the Association for Computational Linguistics*, pages 44–50, Cambridge, Massachusetts, USA, June 1983. Association for Computational Linguistics. doi: 10.3115/981311.981320. URL <https://aclanthology.org/P83-1007>.
- [27] D. Guo, D. Tang, N. Duan, M. Zhou, and J. Yin. Dialog-to-action: Conversational question answering over a large-scale knowledge base. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018. URL <https://proceedings.neurips.cc/paper/2018/file/d63fbf8c3173730f82b150c5ef38b8ff-Paper.pdf>.
- [28] S. Gupta, S. Singh, and M. Gardner. Structurally diverse sampling reduces spurious correlations in semantic parsing datasets. *arXiv*, 2022.

- [29] P. Hase, H. Xie, and M. Bansal. The out-of-distribution problem in explainability and search methods for feature importance explanations. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 3650–3666. Curran Associates, Inc., 2021. URL <https://proceedings.neurips.cc/paper/2021/file/1def1713ebf17722cbe300cfc1c88558-Paper.pdf>.
- [30] S. Hooker, D. Erhan, P.-J. Kindermans, and B. Kim. *A Benchmark for Interpretability Methods in Deep Neural Networks*. Curran Associates Inc., Red Hook, NY, USA, 2019.
- [31] H. Kaur, H. Nori, S. Jenkins, R. Caruana, H. Wallach, and J. Wortman Vaughan. *Interpreting Interpretability: Understanding Data Scientists’ Use of Interpretability Tools for Machine Learning*, page 1–14. Association for Computing Machinery, New York, NY, USA, 2020. ISBN 9781450367080. URL <https://doi.org/10.1145/3313831.3376219>.
- [32] S. Krishna, T. Han, A. Gu, J. Pombra, S. Jabbari, S. Wu, and H. Lakkaraju. The disagreement problem in explainable machine learning: A practitioner’s perspective. *arXiv*, 2022.
- [33] H. Lakkaraju, S. H. Bach, and J. Leskovec. Interpretable decision sets: A joint framework for description and prediction. In *Knowledge Discovery and Data mining (KDD)*, 2016.
- [34] H. Lakkaraju, E. Kamar, R. Caruana, and J. Leskovec. Faithful and customizable explanations of black box models. In *Proceedings of the 2019 AAAI/ACM Conference on AI, Ethics, and Society*, AIES ’19, page 131–138, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450363242. doi: 10.1145/3306618.3314229. URL <https://doi.org/10.1145/3306618.3314229>.
- [35] H. Lakkaraju, D. Slack, Y. Chen, C. Tan, and S. Sing. Rethinking explainability as a dialogue: A practitioner’s perspective. *arXiv*, 2022.
- [36] J. Li, V. Nagarajan, G. Plumb, and A. Talwalkar. A learning theoretic perspective on local explainability. *ICLR*, 2020.
- [37] X. Li, Y.-N. Chen, L. Li, J. Gao, and A. Celikyilmaz. End-to-end task-completion neural dialogue systems. In *the 8th International Joint Conference on Natural Language Processing. IJCNLP 2017*, November 2017. URL <https://www.microsoft.com/en-us/research/publication/end-end-task-completion-neural-dialogue-systems/>.
- [38] Q. V. Liao, D. Gruen, and S. Miller. Questioning the ai: Informing design practices for explainable ai user experiences. *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, 2020.
- [39] Y. Liu, K. Han, Z. Tan, and Y. Lei. Using context information for dialog act classification in DNN framework. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 2170–2178, Copenhagen, Denmark, Sept. 2017. Association for Computational Linguistics. doi: 10.18653/v1/D17-1231. URL <https://aclanthology.org/D17-1231>.
- [40] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.
- [41] Y. Lou, R. Caruana, J. Gehrke, and G. Hooker. Accurate intelligible models with pairwise interactions. In *KDD*, 2013.
- [42] S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, pages 4765–4774, 2017.

- [43] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee. From local explanations to global understanding with explainable ai for trees. *Nature Machine Intelligence*, 2(1):56–67, Jan 2020. ISSN 2522-5839. doi: 10.1038/s42256-019-0138-9. URL <https://doi.org/10.1038/s42256-019-0138-9>.
- [44] C. Meng, L. Trinh, N. Xu, J. Enouen, and Y. Liu. Interpretability and fairness evaluation of deep learning models on mimic-iv dataset. *Scientific Reports*, 12(1):7166, May 2022. ISSN 2045-2322. doi: 10.1038/s41598-022-11012-2. URL <https://doi.org/10.1038/s41598-022-11012-2>.
- [45] S. Min, X. Lyu, A. Holtzman, M. Artetxe, M. Lewis, H. Hajishirzi, and L. Zettlemoyer. Rethinking the role of demonstrations: What makes in-context learning work? In *EMNLP*, 2022.
- [46] R. K. Mothilal, A. Sharma, and C. Tan. Explaining machine learning classifiers through diverse counterfactual explanations. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, pages 607–617, 2020.
- [47] W. J. Murdoch, C. Singh, K. Kumbier, R. Abbasi-Asl, and B. Yu. Definitions, methods, and applications in interpretable machine learning. *Proceedings of the National Academy of Sciences*, 116(44):22071–22080, 2019. doi: 10.1073/pnas.1900654116. URL <https://www.pnas.org/doi/abs/10.1073/pnas.1900654116>.
- [48] I. Oren, J. Herzig, N. Gupta, M. Gardner, and J. Berant. Improving compositional generalization in semantic parsing. In *FINDINGS*, 2020.
- [49] S. Palan and C. Schitter. Prolific.ac—a subject pool for online experiments. *Journal of Behavioral and Experimental Finance*, 17:22–27, 2018. ISSN 2214-6350. doi: <https://doi.org/10.1016/j.jbef.2017.12.004>. URL <https://www.sciencedirect.com/science/article/pii/S2214635017300989>.
- [50] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [51] J. W. Pennebaker, M. R. Mehl, and K. G. Niederhoffer. Psychological aspects of natural language. use: our words, our selves. *Annu Rev Psychol*, 54:547–577, June 2002.
- [52] G. Plumb, D. Molitor, and A. Talwalkar. Model agnostic supervised local explanations. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, NIPS’18, page 2520–2529, Red Hook, NY, USA, 2018. Curran Associates Inc.
- [53] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020. URL <http://jmlr.org/papers/v21/20-074.html>.
- [54] N. Reimers and I. Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 11 2019. URL <http://arxiv.org/abs/1908.10084>.
- [55] M. T. Ribeiro, S. Singh, and C. Guestrin. "why should I trust you?": Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, August 13-17, 2016*, pages 1135–1144, 2016.

- [56] M. T. Ribeiro, S. Singh, and C. Guestrin. Model-Agnostic Interpretability of Machine Learning. In *ICML Workshop on Human Interpretability in Machine Learning (WHI)*, June 2016.
- [57] M. T. Ribeiro, S. Singh, and C. Guestrin. Why Should I Trust You? explaining the predictions of any classifier. In *Knowledge Discovery and Data mining (KDD)*, 2016.
- [58] M. T. Ribeiro, S. Singh, and C. Guestrin. Anchors: High-precision model-agnostic explanations. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), Apr. 2018. doi: 10.1609/aaai.v32i1.11491. URL <https://ojs.aaai.org/index.php/AAAI/article/view/11491>.
- [59] V. Rieser and O. Lemon. *Developing Dialogue Managers from Limited Amounts of Data*, pages 5–17. Springer New York, New York, NY, 2012. ISBN 978-1-4614-4803-7. doi: 10.1007/978-1-4614-4803-7_2. URL https://doi.org/10.1007/978-1-4614-4803-7_2.
- [60] S. Santhanam and S. Shaikh. A survey of natural language generation techniques with a focus on dialogue systems - past, present and future directions. *ArXiv*, abs/1906.00500, 2019.
- [61] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *2017 IEEE International Conference on Computer Vision (ICCV)*, pages 618–626, 2017. doi: 10.1109/ICCV.2017.74.
- [62] Y. Shao, S. Gouws, D. Britz, A. Goldie, B. Strope, and R. Kurzweil. Generating high-quality and informative conversation responses with sequence-to-sequence models. In *EMNLP*, 2017.
- [63] R. Shin, C. H. Lin, S. Thomson, C. Chen, S. Roy, E. A. Platanios, A. Pauls, D. Klein, J. Eisner, and B. V. Durme. Constrained language models yield few-shot semantic parsers. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Punta Cana, Nov. 2021. URL <http://cs.jhu.edu/~jason/papers/#semanticmachines-2021-emnlp>.
- [64] E. Shinan. Lark - a parsing toolkit for python. URL <https://github.com/lark-parser/lark>.
- [65] K. Simonyan, A. Vedaldi, and A. Zisserman. Deep inside convolutional networks: Visualising image classification models and saliency maps. In *Workshop at International Conference on Learning Representations*, 2014.
- [66] D. Slack, A. Hilgard, S. Singh, and H. Lakkaraju. Reliable post hoc explanations: Modeling uncertainty in explainability. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 9391–9404. Curran Associates, Inc., 2021. URL <https://proceedings.neurips.cc/paper/2021/file/4e246a381baf2ce038b3b0f82c7d6fb4-Paper.pdf>.
- [67] D. Slack, N. Rauschmayr, and K. Kenthapadi. Defuse: Training more robust models through creation and correction of novel model errors. *XAI4Debugging Workshop @ NeurIPS*, 2021. URL https://xai4debugging.github.io/files/papers/defuse_training_more_robust_mo.pdf.
- [68] D. Smilkov, N. Thorat, B. Kim, F. Viégas, and M. Wattenberg. Smoothgrad: removing noise by adding noise. *Workshop on Visualization for Deep Learning, ICML*, 2017.
- [69] K. Sokol and P. A. Flach. Glass-box: Explaining ai decisions with counterfactual statements through conversation with a voice-enabled virtual assistant. In *IJCAI*, pages 5868–5870, 2018.
- [70] M. Sundararajan, A. Taly, and Q. Yan. Axiomatic attribution for deep networks. In *Proceedings of the 34th International Conference on Machine Learning - Volume 70, ICML’17*, page 3319–3328. JMLR.org, 2017.

- [71] I. Sutskever, O. Vinyals, and Q. V. Le. Sequence to sequence learning with neural networks. In *Proceedings of the 27th International Conference on Neural Information Processing Systems - Volume 2*, NIPS’14, page 3104–3112, Cambridge, MA, USA, 2014. MIT Press.
- [72] A. Talmor, M. Geva, and J. Berant. Evaluating semantic parsing against a simple web-based question answering model. In *Proceedings of the 6th Joint Conference on Lexical and Computational Semantics (*SEM 2017)*, pages 161–167, Vancouver, Canada, Aug. 2017. Association for Computational Linguistics. doi: 10.18653/v1/S17-1020. URL <https://aclanthology.org/S17-1020>.
- [73] I. Tenney, J. Wexler, J. Bastings, T. Bolukbasi, A. Coenen, S. Gehrmann, E. Jiang, M. Pushkarna, C. Radebaugh, E. Reif, and A. Yuan. The language interpretability tool: Extensible, interactive visualizations and analysis for NLP models, 2020. URL <https://www.aclweb.org/anthology/2020.emnlp-demos.15>.
- [74] B.-H. Tseng, S. Bhargava, J. Lu, J. R. A. Moniz, D. Piraviperumal, L. Li, and H. Yu. CREAD: Combined resolution of ellipses and anaphora in dialogues. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3390–3406, Online, June 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.naacl-main.265. URL <https://aclanthology.org/2021.naacl-main.265>.
- [75] B. Ustun and C. Rudin. Supersparse linear integer models for optimized medical scoring systems. *Mach. Learn.*, 102(3):349–391, 2016. doi: 10.1007/s10994-015-5528-6. URL <https://doi.org/10.1007/s10994-015-5528-6>.
- [76] B. Wang and A. Komatsuzaki. GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model. <https://github.com/kingoflolz/mesh-transformer-jax>, May 2021.
- [77] N. G. Ward and D. DeVault. Ten challenges in highly-interactive dialog systems. In *AAAI 2015*, 2015.
- [78] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, E. H. Chi, T. Hashimoto, O. Vinyals, P. Liang, J. Dean, and W. Fedus. Emergent abilities of large language models. *arXiv*, 2022.
- [79] D. S. Weld and G. Bansal. The challenge of crafting intelligible intelligence. *Commun. ACM*, 62(6):70–79, may 2019. ISSN 0001-0782. doi: 10.1145/3282486. URL <https://doi.org/10.1145/3282486>.
- [80] S. M. Xie, A. Raghunathan, P. Liang, and T. Ma. An explanation of in-context learning as implicit bayesian inference. *ICLR*, 2022.
- [81] C.-K. Yeh, C.-Y. Hsieh, A. Suggala, D. I. Inouye, and P. K. Ravikumar. On the (in) fidelity and sensitivity of explanations. In *Advances in Neural Information Processing Systems*, pages 10965–10976, 2019.
- [82] P. Yin, H. Fang, G. Neubig, A. Pauls, E. A. Platanios, Y. Su, S. Thomson, and J. Andreas. Compositional generalization for neural semantic parsing via span-level supervised attention. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2810–2823, Online, June 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.naacl-main.225. URL <https://aclanthology.org/2021.naacl-main.225>.

- [83] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Brussels, Belgium, 2018. Association for Computational Linguistics.
- [84] J. Zeng, B. Ustun, and C. Rudin. Interpretable classification models for recidivism prediction. *Journal of the Royal Statistical Society: Series A (Statistics in Society)*, 180, 03 2015. doi: 10.1111/rssa.12227.
- [85] X. Zhang, S. Tan, P. Koch, Y. Lou, U. Chajewska, and R. Caruana. Axiomatic interpretability for multiclass additive models. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '19, page 226–234, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450362016. doi: 10.1145/3292500.3330898. URL <https://doi.org/10.1145/3292500.3330898>.
- [86] Z. Zhang, R. Takanobu, Q. Zhu, M. Huang, and X. Zhu. Recent advances and challenges in task-oriented dialog systems. *Science China Technological Sciences*, 63(10):2011–2027, Oct. 2020.
- [87] T. Z. Zhao, E. Wallace, S. Feng, D. Klein, and S. Singh. Calibrate before use: Improving few-shot performance of language models. In *International Conference on Machine Learning*, 2021.

5 Broader Impact Statement

The TalkToModel system and, more generally, conversational model explainability can be applied to a wide range of applications, including financial, medical, or legal applications. Our research could be used to improve model understanding in these situations by improving transparency and encouraging the positive impact of ML systems, while reducing errors and bias. Although TalkToModel has many positive applications, the system makes it easier for those without high levels of technical expertise to understand ML models, which could lead to a false sense of trust in ML systems. In addition, because TalkToModel makes it easier to use ML model for those with lower levels of expertise, there is additionally a risk of inexperienced users applying ML models inappropriately. While TalkToModel includes several measures to prevent such risks, such as qualifying when explanations or predictions are inaccurate, and clearly describing the intended purpose of the ML model, it would be useful for researchers to investigate and possible adopters to be mindful of these considerations.

6 Acknowledgements

The authors would like to acknowledge helpful feedback from Peter Hase, Johan Ugander, Marco Tulio Ribeiro, Brian Lim, and the UCI NLP lab concerning earlier versions of the system and manuscript. This work is supported in part by the NSF awards #IIS-2008461, #IIS-2040989, #IIS-2046873, #IIS-2008956, and research awards from Google, JP Morgan, Amazon, Harvard Data Science Initiative, the D^3 Institute at Harvard, and the Hasso Plattner Institute. H.L. would like to thank Sujatha and Mohan Lakkaraju for their continued support and encouragement. The views expressed here are those of the authors and do not reflect the official policy or position of the funding agencies.

7 Contributions

D.S. designed and developed the TalkToModel system, studies, and prepared the manuscript. S.K. designed and implemented the explanation selection procedure, drafted sections in the manuscript, and edited the manuscript. S.S. and H.L. contributed equally to advising the development of the system and experiments, conceiving the system, reviewing the manuscript, and editing the manuscript.

8 Competing Interests

The authors do not declare any competing interests.

Supplementary Information

A Experimental Details

In this appendix, we provide an overview of additional experimental details. First, we provide additional details about the grammar. Next, we describe how we generate our parsing training sets. Further, we provide more details about the explanation selection procedure. After, we provide examples of the sentences we had mturk workers revise to create our gold parsing datasets. Finally, we provide additional details about the user study.

A.1 Grammar Details

In this subsection, we provide additional details about the grammar. First, we describe the design of the grammar. After, we provide details about how we update the grammar for new datasets.

Design Recall from the main paper that the grammar serves as a logical form of user utterances, which TalkToModel can execute. Here, we provide more details about the grammar design. The grammar defines relations between the **operations** and the acceptable values **arguments** in Table 2. For example, the grammar defines different acceptable values for the **comparison** argument in the **filter** operation, such as **less than or equal to** or **greater than**. In addition, we structure the grammar to make parses appear closer to natural language text instead of a formal programming language like SQL or Python. The reason is that language models tend to perform better at translating utterances into grammars that are more similar to natural language instead of a programming language [63]. Consequently, we design the grammar, so that parses appear more like natural language text, without unnecessary parentheses and commas, and omitting unnecessary arguments where possible. In general, we found that simplifying the grammar and making it more like natural language as much as possible considerably improved performance. For example, the question “What are the three most important features for people older than thirty-five?” would simply translate to **filter age greater than 35 and topk 3**. Note, here, because TalkToModel is applied to only one dataset at a time, the **dataset** argument can be omitted for simplicity. Practically, we implement the grammar in Lark [64] because the implementation supports interactive parsing, simplifying the process of implementing the guided decoding strategy.

Updating the Grammar For Datasets and User Utterances In the main paper, we discussed how we update the grammar based on the dataset. Here, we provide more description about how we update the grammar. We update the grammar based on the feature names and categorical feature values in the dataset. In particular, the acceptable values for the **feature** argument (Table 2) in the grammar becomes all the feature values in the dataset. Further, the **value** argument for a categorical **feature** becomes the set of categorical feature values that appear in the data for that feature. Because there are many possible values of numeric features, we instead extract potential numeric values from user utterances as they are provided to the system. Specifically, we set the **value** argument in the grammar for numeric features to contain the set of numeric values that appear in the user utterance. We additionally support string based numeric values (e.g., “fifty-five” or “twelve”) to make the system handle a wider variety of cases.

A.2 Training Dataset Generation

In this subsection, we provide details about the generation of the (utterance, parse) pair training dataset. To ensure that we generate a diverse and comprehensive set of (utterance, parse) pairs for training, we compose a total of 687 templates that use 6 different wildcard types. The templates consist of a diverse set of utterances that encompass the different operations permitted in the system. The wildcards include categorical feature names, numeric feature names, class names, numeric feature values, categorical feature values, explanation types, and common filtering expressions (e.g., “{NUMERIC_FEATURE} above {NUMERIC_VALUE}”). Because templates can have potentially many wildcards, we recursively enumerate all the wildcard values for each parse. Further, we also limit the number of values for certain wildcards to ensure the number of training pairs generated does not become extremely large. In particular, we limit the number of numeric values to 2 values per feature. In addition, to prevent templates with many wildcards from dominating the training dataset, we also downsample the number of values per wildcard to 2 values after the initial recursion. In this way, we the training dataset does not get dominated by a few templates that have many wildcards, which we found improves performance.

As an example, an utterance template is “Explain the predictions on data with {NUMERIC_FEATURE} greater than {NUMERIC_VALUE}” and the corresponding parse template is `filter {NUMERIC_FEATURE} greater than {NUMERIC_VALUE} and explain feature importance`. From there, we enumerate the numeric features in the dataset and a selection of numeric feature values, substituting these into {NUMERIC_FEATURE} and {NUMERIC_VALUE} respectively to generate data.

A.3 Gold Parsing Dataset

In this subsection, we provide examples of 10 of the sentences provided to mturk workers to revise during our data generation process for each dataset. The examples are provided in Table 1 and illustrate the different types of utterances revised by mturkers to create a comprehensive testing set. Also, we provide the number of (utterance, parse) pairs in the IID and compositional splits in Table 2.

Supplementary Table 1: Examples of 10 sentences out of 50 from each dataset provided to mturk workers to revise to generate parsing evaluation data.

COMPAS	What is your reasoning for determining if people older than 20 are likely to commit crimes?
	How likely are people that are younger than 25 or have committed at least 1 crime in the past to commit a crime in future?
	what are top 3 most important features you use for prediction for people if they were to decrease their prison terms by 10 months?
	For people that are 18 years old and black, how often are you correct in predicting whether they will commit crimes in the future?
	let's look at those in the data with 3 or more prior crimes on record. what are some common mistakes the model makes on these people?
	For this subset in the data, how accurate is the model?
	For people that are 18 years old and black, how often are you correct in predicting whether they will commit crimes in the future?
	how likely would the person with the id number of 33 in the data be to a commit a crime if they were 5 years younger?
	But what if they were twenty years older?
	Could you show me some data for people who are black?
Diabetes	How likely are people that either (1) have had two pregnancies or (2) are older than 20 and younger than 30 to have diabetes?
	What are the top five most important features for the model's predictions on people with a bmi over 40?
	Show the data for people older than 20. Then, could you show me the predictions on this data?
	what would happen to the likelihood of having diabetes if we were to increase glucose by 100 for the data point with id 33
	What's the average age in the data?
	For this subset in the data, how accurate is the model?
	What does patient number 34 need to do in order to be diagnosed as unlikely to have diabetes?
	What are the reasons why the model predicted data point number 100 and what could you do to change it?
German	How would the predictions change if age were decreased by 5 years for people with a bmi of 30?
	How do the features of the data interact in the model's predictions on this particular data?
	If people in the data were unemployed, how important would the age and loan amount features be for predicting credit risk?
	what would happen to the likelihood of being bad credit risk if we were to increase the loan amount by 250 for the data point with id 89
	what are top three most important features for determining whether those who are applying for furniture loans are good credit risk?
	What is the average loan amount for people with no current loans and that do not own a house?
	How accurate are you at predicting whether people who are asking for loans for home appliances are good credit risk?
	In the dataset, if the loan duration were to be increased by 2 years, what would the predictions for the data be?
	Could you show me some examples the model predicts incorrectly and how accurate the model is on the data?
	What do you predict on the instances in the data? Also, could you show me an example of a few mistakes you make in these predictions?
	But why did you think these people are bad credit risk?
	If these people were not unemployed, what's the likelihood they are good credit risk? Why?

Supplementary Table 2: The number of gold (utterance, parse) pairs in the IID and Compositional splits for the datasets. There are relatively more IID questions in each dataset.

	COMPAS	Diabetes	German
IID	117	147	127
Compositional	29	43	74
Overall	146	190	201

A.4 User Study Questions

In this subsection, we provide the questions participants answered in the user study. The questions are provided in Table 3. One of the two question blocks, Block 1 or Block 2, is shown for TalkToModel and the other is shown for the dashboard (the ordering of TalkToModel and the dashboard is also randomized). The two question blocks include similar concepts to ensure a similar level of difficulty but include different numbers to discourage memorization between the question blocks.

Supplementary Table 3: The tasks solved by participants during the user study. Participants were shown either one block for TalkToModel and the other block for the dashboard.

Block 1	What are the three most important features for the model's predictions on people older than 30 with bmi's above 35?
	What is the feature importance ranking of the age feature for data point id 188?
	How many individuals in the dataset are predicted to be likely to have diabetes but are not actually likely to have it?
	If patient id 293 were to decrease their bmi by five, what's the prediction probability of the "likely to have diabetes" class?
	Is the "glucose" feature more important than the "age" feature for the model's prediction on data point 49?
Block 2	What are the three most important features for the model's predictions on people younger than 23 with glucose levels below seventy-five?
	What is the feature importance ranking of the insulin feature for data point id 57?
	How many individuals in the dataset are predicted not likely to have diabetes but actually are likely to have it?
	What's the likelihood of patient 57 having diabetes if they increased their glucose levels by 100 and bmi by 3?
	How important is the "diabetes pedigree function" feature compared to the "glucose" feature for the model's prediction on data point 55?

A.5 User Study Length & Compensation

The survey took around 30 minutes for participants to complete. We compensated the ML professionals by providing them with a \$20 gift card. We paid the prolific workers \$14.74/hour on average for completing the survey, considering their individual completion times.

B Additional Experiments

In this appendix, we provide additional experimental results. First, we provide experimental results where we show the advantages of explanation selection. Second, we give results on the affects of the number of training templates. After, we provide error analysis for our parsing models. Last, we give additional user study results.

B.1 Advantages of Explanation Selection

In the main paper, we introduced a technique for explanation selection, which we used in TalkToModel to automatically select high-quality explanations for the conversation (Subsection 4.2). In this appendix, we provide more details about the advantages of explanation selection. In particular, we rigorously benchmark our selection method against SOTA explanation techniques like LIME and SHAP. We show that our explanation selection method computes the most faithful explanations.

Supplementary Table 4: The prediction gap on important features (PGI) and prediction gap on unimportant features (PGU) results. We bold the statistically significant best result. Overall, explanation selection is the best explanation method in all settings, except for PGU and the german credit data where it is better than SHAP but not significantly better than LIME.

	PGI $\uparrow$			PGU $\downarrow$		
	Diabetes	COMPAS	German	Diabetes	COMPAS	German
LIME, Width=0.25	0.070	0.124	3.897	0.032	0.031	0.774
LIME, Width=0.50	0.072	0.127	3.871	0.020	0.027	0.793
LIME, Width=0.75	0.071	0.127	3.856	0.021	0.026	0.808
LIME, Width=1.0	0.070	0.127	3.853	0.022	0.026	0.799
SHAP	0.083	0.117	2.094	0.031	0.031	3.007
Explanation Selection	0.107	0.138	4.011	0.006	0.023	0.788

To perform this analysis, we use the faithfulness metrics provided by the widely-used OpenXAI framework [4]. Specifically, we use the Prediction Gap on Important feature perturbation (PGI) and the Prediction Gap on Unimportant feature perturbation (PGU) metrics. These metrics measure the change in perturbing the most influential features and least important features, respectively. Intuitively, PGI captures that perturbations to influential features should result in more significant changes to predictions (higher PGI is better). PGU captures that perturbations to non-influential features should result in smaller changes to the prediction (lower PGU is better).

We compare our explanation selection method against both SOTA explanation methods LIME and SHAP [55, 42]. To make our evaluation more comprehensive, we compare against LIME using 4 different settings of the kernel width hyperparameter [0.25, 0.50, 0.75, 1.0], because this hyperparameter can have significant effects on the resulting explanation. We leave all settings to default otherwise. Further, we perform this comparison using our 3 diverse datasets: Diabetes, COMPAS, and German Credit, and we compute explanations three times for each data point to reduce error due to explanation sampling. We set the important features used for the PGI metric to the most influential 50% of features and the unimportant features used for the PGU metric to the least influential 50% of features to ensure we provide a comprehensive evaluation for the explanation’s ranking of all features in the data.

We present the results in Table 4 and provide the mean PGU or PGI value for each explanation and dataset. Further, we bold the best statistically significant result according to a Bonferroni corrected t-test, where we compare the explanation selection procedure to each of the other explanation methods for the respective dataset and metric. Overall, we find that explanation selection performs better than baseline SOTA explanations across almost every dataset and metric considered, except for the PGU metric on the German dataset, where explanation selection performs on par with the best performing LIME explanations.

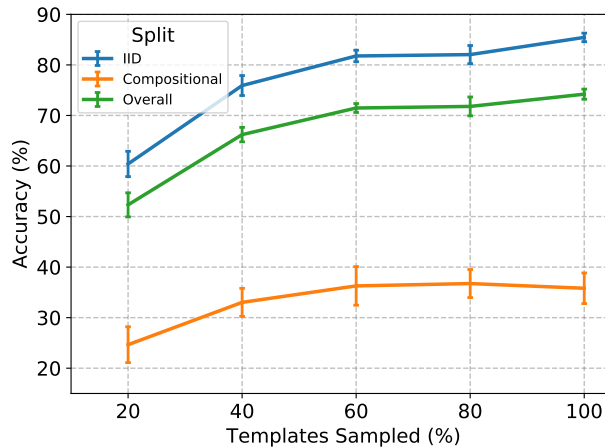
B.2 Effects of the number of training templates

In this subsection, we provide experimental details about the effects of the number of training templates on the parsing accuracy of the system. Because we use a template strategy to generate training data (Section 4), we must decide on how many prompts to write and include in the training scheme. This raises the question of how the number of templates affects model performance. To understand this behavior, we retrain the T5-Base model, randomly sampling the number of training templates over different percentages of the original templates set. In particular, we sweep over the following percentages [20%, 40%, 60%, 80%, 100%] on the diabetes dataset, downsampling and retraining 5 times

	5-Shot		10-Shot	
	IID	Compositional	IID	Compositional
Compas	29.8	77.8	15.3	53.6
Diabetes	40.0	40.0	19.1	28.2
German	44.2	50.8	28.3	26.9

Supplementary Table 5: Percentage of mistakes for few-shot GPT-J 6B where selected prompts *do not* include all the operations in the parse of the user utterance. We see that most of the time the operations for the parse of the user utterance are included in the prompts for the 10-Shot models, yet these methods still perform relatively poorly compared to finetuned T5.

per template. We give the results in Figure 1. We see that there are clear accuracy gains over using a smaller number of templates. Further, the gains for the compositional model performance seem to somewhat level off, but this is not the case for the IID split, suggesting further templates may help IID performance.



Supplementary Figure 1: Randomly sampling prompt templates and re-training T5-Base on the Diabetes dataset. For each down-sample %, the prompts are randomly down-sampled and the model is re-trained 5 times. The error bars are 1 standard deviation.

B.3 Parsing Error Analysis

In the main text, we demonstrated that the fine-tuned T5 models performed considerably better than few-shot GPT-J (Table 3). In this subsection, we perform additional error analysis for why this occurs. This dynamic brings up the question: what is the cause of these poor few-shot results? Since the few-shot GPT-J models select the (utterance, parse) pairs from the synthetic dataset using nearest neighbors on a sentence embedding model, it could be possible these issues are due to the sentence embedding model failing to select good pairs. In particular, this nearest neighbors technique could fail to select pairs with the operations necessary to parse the user utterance, and not the model failing to learn from the examples. To evaluate whether this is the case, we compute the percent of mistakes that *do not* include the operations necessary to parse the user utterance for GPT-J. The results provided in Table 5 demonstrate that, especially for the 10-Shot case, the operations needed to parse the user utterance *are* included in the prompts, indicating the issues are likely due to the model’s capacity to

learn few-shot, rather than the selection mechanism. In this work, we were limited by using up to 6-billion parameter GPT-J, but it could be possible to achieve better results with larger models, as results on emergent abilities suggest [78].

B.4 User Study Results: Per Question Likert Scores

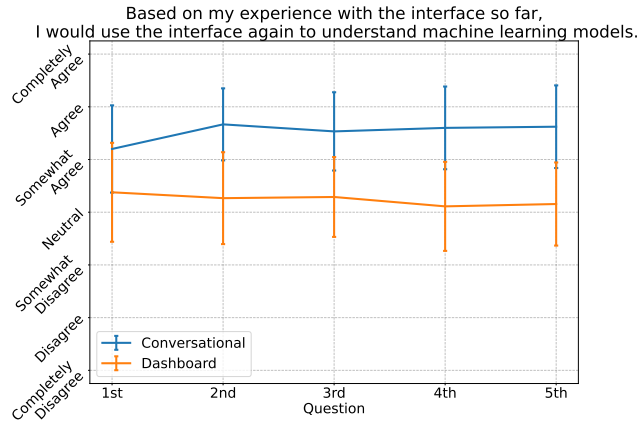
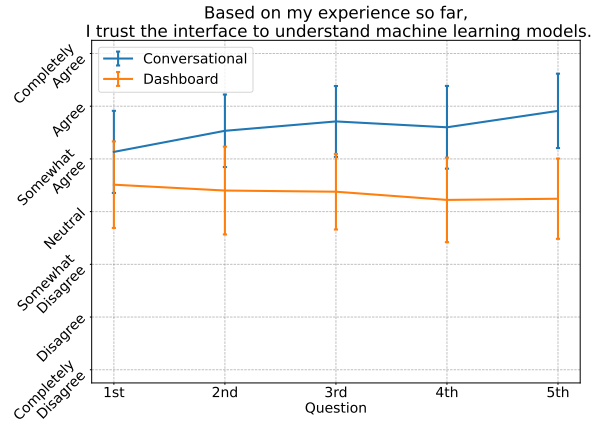
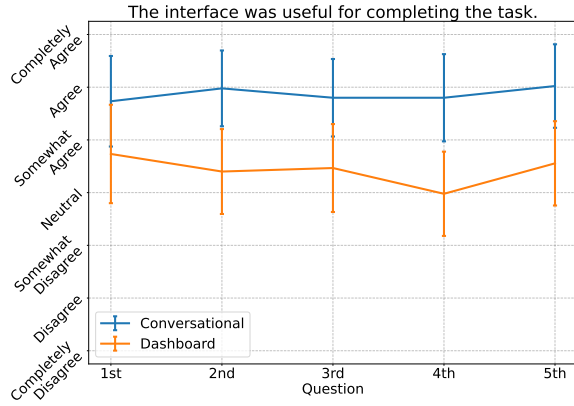
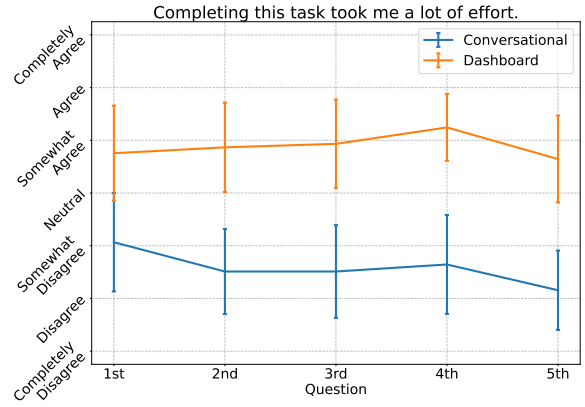
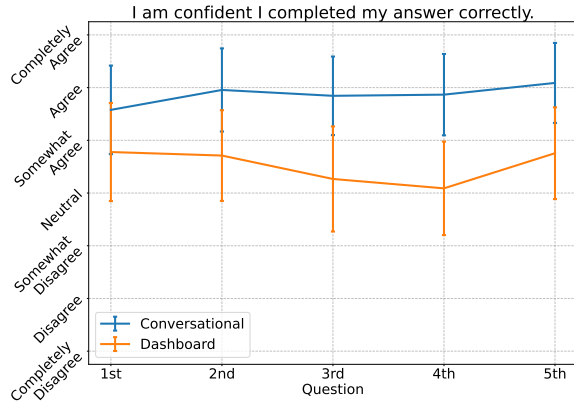
In this subsection, we provide additional user study results. In addition to the questions asked at the end of the survey (Table 4 and Table 5), we also asked users to rate their experiences using both interfaces on a 1-7 Likert while they were taking the survey. In particular, we asked users how much they agreed with the following statements:

- I am confident I completed my answer correctly.
- Completing this task took me a lot of effort.
- The interface was useful for completing the task.
- Based on my experience so far, I trust the interface to understand machine learning models.
- Based on my experience so far, I would use the interface again to understand machine learning models.

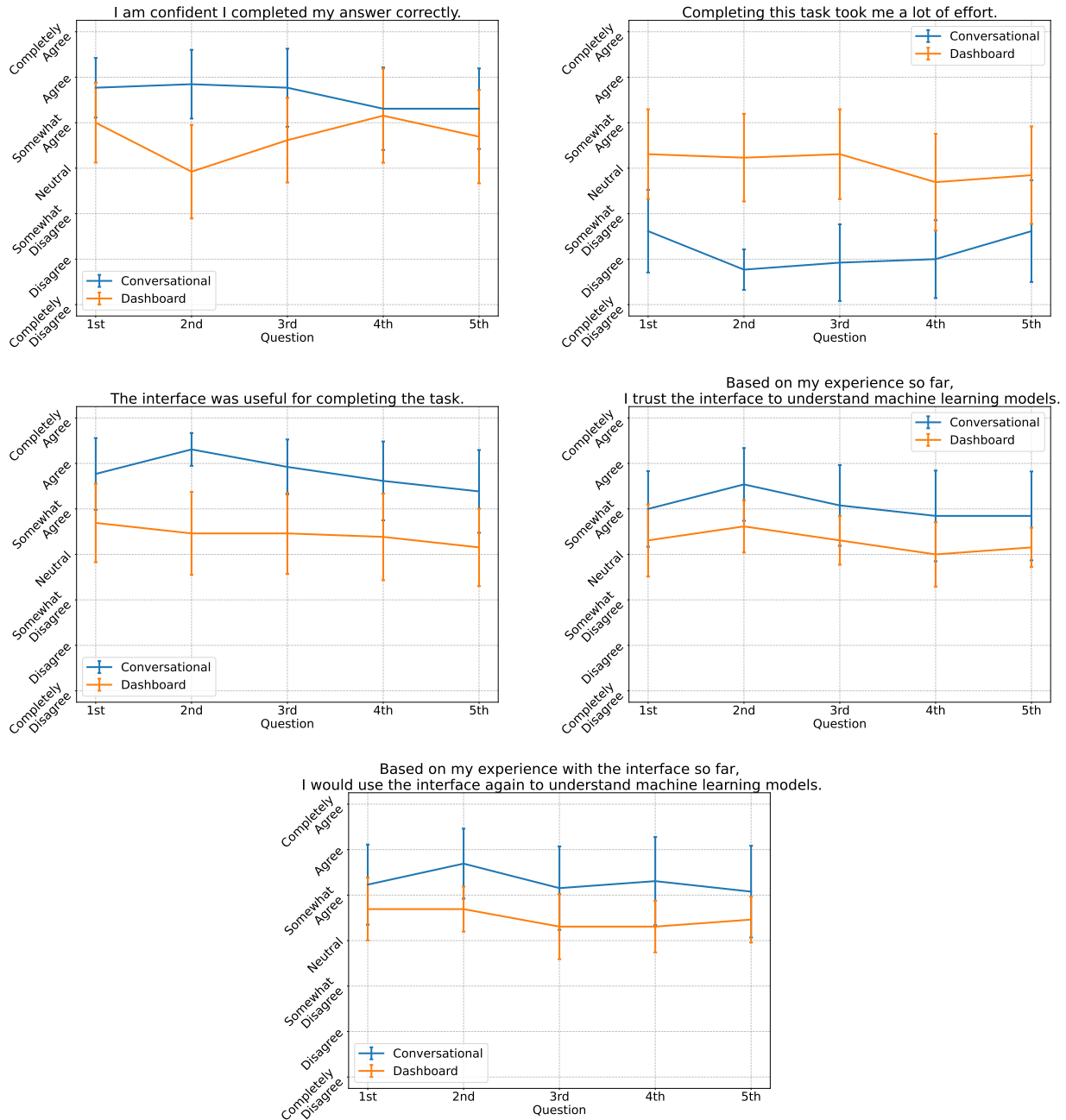
To evaluate these results, we compute the mean and standard deviation of the Likert score for the 1st through 5th question each user sees (question ordering is randomized so users see different questions first). We compute this for each statement and interface. The results for the medical workers are provided in Figure 2 and the ML professionals in Figure 3. Overall, the medical workers clearly prefer TalkToModel while answering the questions. Interestingly, they seem to gain trust in TalkToModel over time, going from “somewhat agree” to “agree” with the statement “Based on my experience so far, I trust the interface to understand ML models” by the end of the survey.

B.5 XAI Question Bank

Here, we provide parses in the TalkToModel for the prototypical questions given in the XAI question bank [38]. Our grammar can parse 30/31 core, prototypical questions, excluding socio-technical questions, demonstrating the grammar’s broad coverage. Note, that questions provided in the question bank vary in how they are phrased regarding whether additional coreference is necessary. For instance, the question bank includes both questions of the form “what do you predict for this?” versus “what do you predict for Q?”. For conciseness, we write each question in the form where no further coreference is necessary (“what do you predict for Q?”). For the case where additional coreference is necessary it is straightforward to use the `previous_filter` operation to resolve the coreference. These results demonstrate the TalkToModel grammar is well equipped to support XAI questions.



Supplementary Figure 2: **Medical Worker per question likert results:** in general, these participants preferred TalkToModel over the dashboard to answer the questions. Error bars are 1 standard deviation.



Supplementary Figure 3: **ML professionals per question likert results:** these participants were more mixed about which interface they preferred while taking the survey. Interestingly, while the participants were much more accurate using TalkToModel, this population rated their answers at a similar confidence and said they trusted the interfaces similarly while taking the survey. Error bars are 1 standard deviation.

Supplementary Table 6: Prototypical questions from the XAI question bank, parses in the TalkToModel grammar, and explanations of the parse.

What kind of output does the system give?

`function()`

System overview describes model output.

What does the system output mean?

`function()`

System overview describes meaning of model outputs (e.g., predict if someone has diabetes).

What kind of data was the system trained on?

`data(training_data)`

Data overview provides summary of dataset.

What is the sample size of the training data?

`count(training_data)`

Count provides number of items in the training data.

What is the distribution of the training data with with a given feature?

`statistic(training_data, feature_name)`

Statistic summarizes feature distribution

How accurate are the predictions?

`score(test_data, accuracy)`

Scoring functionality gets accuracy on test set

How often does the system make mistakes?

`incorrect(test_data, accuracy)`

Shows how often the model makes incorrect predictions.

In what situations is the system likely to be incorrect?

`mistakes(test_data)`

Summarizes the common situations mistakes are made and ways the system is wrong.

What kind of mistakes is the system likely to make?

`mistakes(test_data)`

Summarizes the common situations mistakes are made and ways the system is wrong.

How does the system make predictions?

`explain(test_data, feature_importance)`

Provides an overview of features used for making predictions

What features does the system consider?

`topk(test_data, all)`

Shows the feature ranking to demonstrate which features the model uses.

What would the system predict if a given feature A changes to..?

`predict(change(filter(test_data, id, A), feature, value, set))`

Shows predictions under single feature change.

Is feature X used or not used for the predictions

`important(test_data, X)`

Determines whether feature X is important for the prediction.

How should a given feature A change for this instance to get a different prediction Q?

`statistic(cfe(filter(test_data, id, Q, =), 100), summary, A)`

Summarizes changes to feature that will flip prediction.

What is the systems overall logic?

`explain(test_data, feature_importance) interaction(test_data)`

Provides first order feature importances and second order interaction effects to explain overall system logic.

Supplementary Table 7: Prototypical questions from the XAI question bank, parses in the TalkToModel grammar, and explanations of the parse (continued).

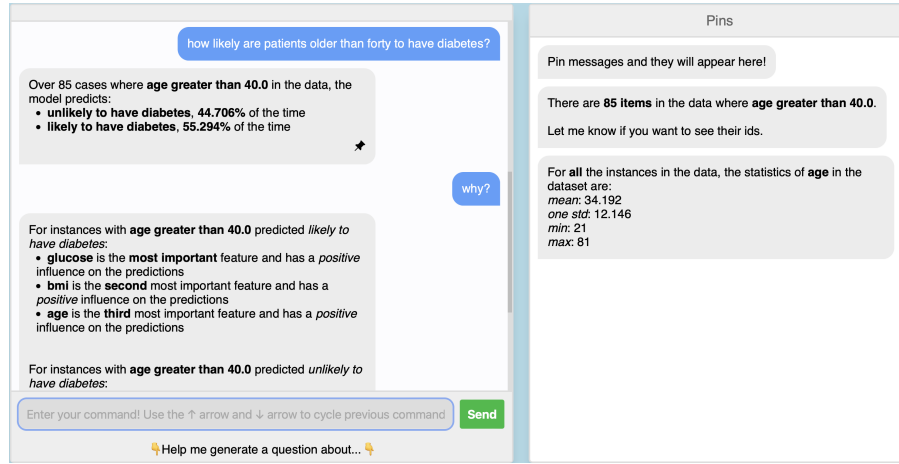
What features of instance Q determine the system's prediction of it?
<code>topk(filter(test_data, id, Q, =), all)</code>
Shows most important features that determine prediction.
Why are instance A and B given the same prediction?
<code>explain(or(filter(test_data, id, A, =), filter(test_data, id B, =))), feature_importance)</code>
Explains predictions for both instances by summarizing shared most important features across instances.
Why is this instance not predicted to be Q?
<code>explain(filter(text_data, id, A, =), feature_importance, class=Q)</code>
Explains alternate class prediction.
Why are instance A and B given different predictions?
<code>statistic(or(explain(filter(text_data, id, A, =), feature_importance), explain(filter(text_data, id, B, =), feature_importance)), range, all)</code>
Explanations reason for different prediction by contrasting feature importances.
How should instance A change to get a different prediction Q?
<code>cfe(filter(test_data, id, A, =), 10, Q)</code>
Computes several counterfactual explanations to provide different ways to get alternate predictions.
What kind of algorithm is used?
<code>model()</code>
Describes the ML model.
Why is the instance given this prediction?
<code>explain(filter(test_data, id, X, =), feature, feature_importance)</code>
Explains prediction for given instance with feature importance.
What is the minimum change required for instance A to get a different prediction Q?
<code>cfe(filter(test_data, id, A, =), 1, Q)</code>
Computes single, minimal counterfactual.
What kind of instance is predicted outcome Q?
<code>statistic(filter(test_data, Y, Q, =), summary, all)</code>
Summarizes instances with certain predicted outcome.
What is the scope of change permitted for instance A to still get the same prediction?
<code>statistic(cfe(filter(test_data, id, A, =), 100), min, all)</code>
Summarizes minimal changes to flip prediction.
What is the range of values permitted to for a given feature for this prediction on A to stay the same?
<code>statistic(cfe(filter(test_data, id, A, =), 100), min, feature)</code>
Summarizes minimal changes to certain feature that will flip prediction.
What kind of instance gets the same prediction Q?
<code>statistic(filter(test_data, Y, Q, =))</code>
Summarizes instances with the same prediction Q.
What would the system predict if instance A changes to...?
<code>predict(and(change(filter(test_data, id, A), feature, value, set), change(filter(test_data, id, A), feature, value, set)...))</code>
Shows predictions under potentially many different changes to instance.
What would the system predict for [a different instance A]?
<code>predict(filter(test_data, feature, A, =))</code>
Shows predictions for different instances according to filtering criteria.
What does [a machine learning terminology] mean?
<code>define(term)</code>
Defines a term.

C Interface Screenshots

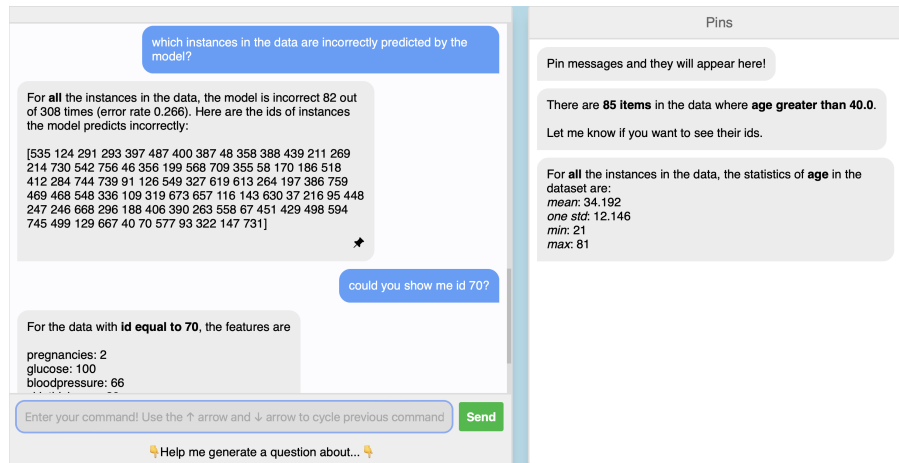
In this appendix, we provide additional interface screen shots for both TalkToModel and the dashboard baseline.

C.1 TalkToModel Interface

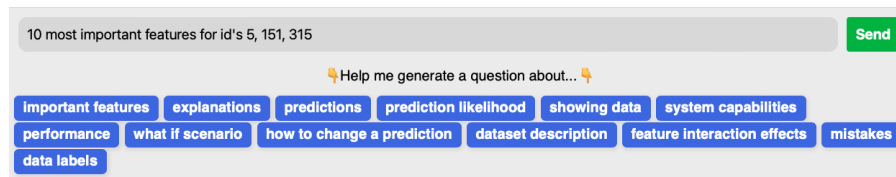
We provide additional screenshots of the TalkToModel GUI interface in Fig. 4 and Fig. 5. The interface is a text chat interface, where users provide input and the system response. On the right side of the interface, users can “pin” messages they find interesting. In addition, we provide a feature where users can get help to generate a question in a particular category (Fig. 6). This feature inputs a random synthetic training utterance into the command bar from the category of choice.



Supplementary Figure 4: Discussing the model’s predictions on patients older than 40. The system correctly understand’s the context of the question “why?”.



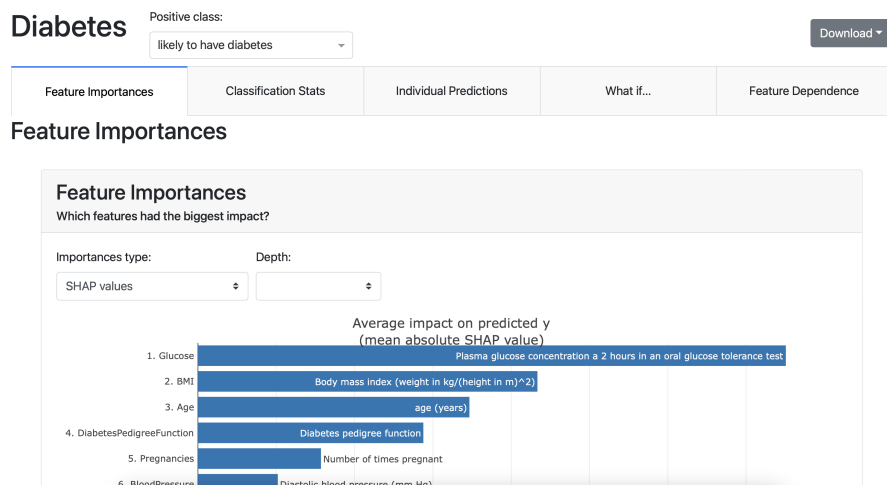
Supplementary Figure 5: Beyond explanations, TalkToModel is also highly useful for classic error analysis, such as inspecting incorrectly predicted instances, as is shown here.



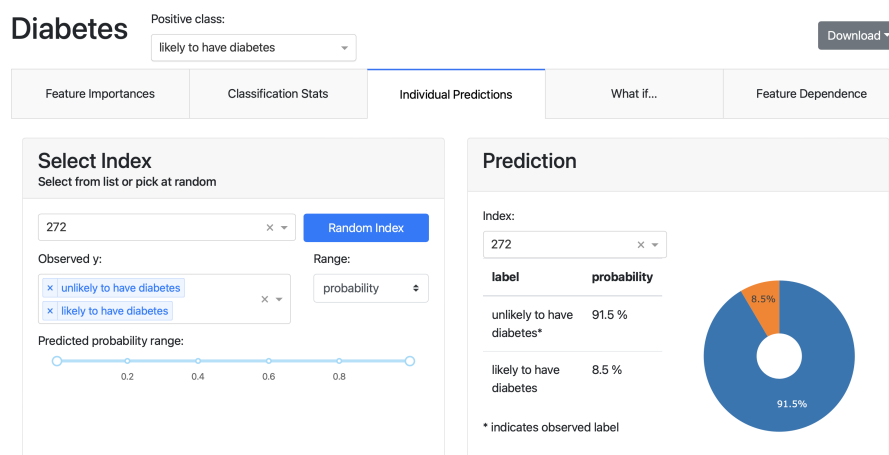
Supplementary Figure 6: Screen shot of the help generate a question button, which will input a random question from the training data in the category when clicked..

C.2 Dashboard Interface

We provide example screenshots for the baseline dashboard interface in Fig. 7 and Fig. 8 [18]. In depth description can be found in the project’s documentation <https://explainerdashboard.readthedocs.io/en/latest/>. The dashboard provides different ways to understand ML models using point and click interactions.



Supplementary Figure 7: The dashboard interface lands on the global feature importances. Users can navigate to other tabs in the interface to see model predictions, metrics, feature importances, and compute what-if scenarios.



Supplementary Figure 8: An example of the “individual predictions” page in the dashboard interface. Users need to navigate to this page and type in the data point index they want in order to inspect predictions.